Module 10: Autonomous Agents

Plan, execute, verify, report

Module 10: Autonomous Agents

Primary sources: Osmani, Chapter 10; Thomas & Hunt, Topics 41-42; Topic 51: Pragmatic Starter Kit, section "Ruthless and Continuous Testing."

Learning Outcomes

- Evaluate capabilities and limitations of autonomous agents.
- Compare agent-based and traditional workflows.
- Assess risks of increased autonomy.
- Justify appropriate use of agents.

From Copilot to Agent

In-IDE assistants operate under close human supervision.

Autonomous or background agents take a larger task and work through multiple steps.

This shifts the main question from "Can it write code?" to "Can we safely delegate this workflow?"

Background Agent Model

Osmani describes a common background-agent model: asynchronous assistant developers working in controlled environments.

- They receive a high-level task.
- They clone or inspect a repository in an isolated environment.
- They plan changes across files.
- They edit, run commands, and test.
- They return a diff, branch, pull request, or report for human review.

The agent is not just completing code; it is operating inside the development workflow.

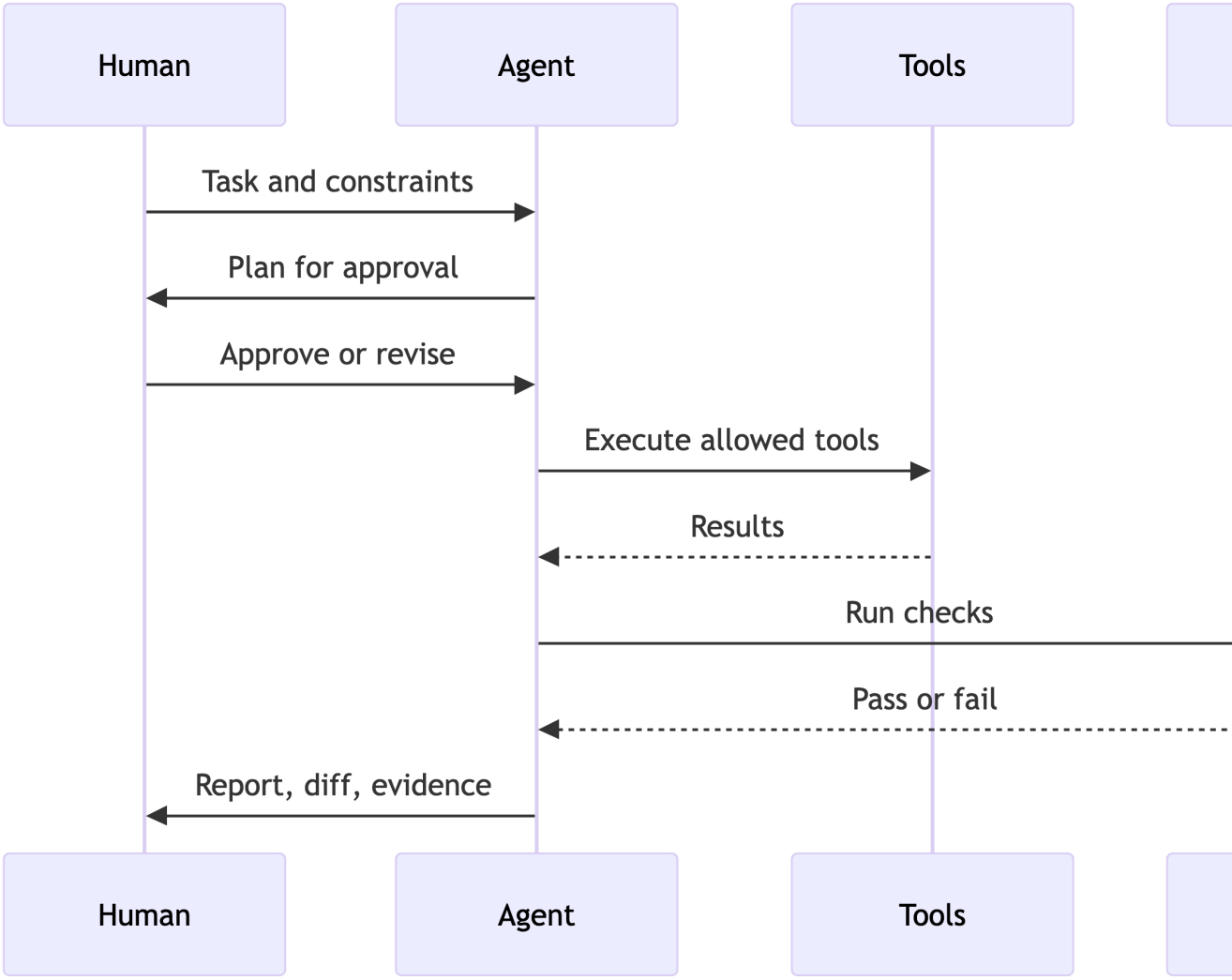
Agent Loop

Common autonomous workflow:

1. Plan.
2. Execute.
3. Verify.
4. Report.
5. Receive human feedback.

Your course agent should make this loop inspectable.

Agent Control Points



Appropriate Tasks

Good candidates:

- Small, well-scoped changes.
- Clear tests or checks.
- Low-risk refactors.

- Documentation updates.
- Repetitive maintenance.

Poor candidates:

- Ambiguous architecture decisions.
- Security-critical changes without review.
- Large rewrites.
- Changes with weak verification.

Agent vs IDE Assistant

Autonomous agents and traditional assistants solve different problems.

- IDE assistants are synchronous, local, and closely supervised.
- Background agents are asynchronous, project-level, and delegated.
- IDE assistants are better for tight-control logic and exploration.
- Agents are better for bounded maintenance, cross-file chores, and repetitive updates.
- Agent output shifts human effort from writing to reviewing and deciding.

Autonomy Risks

- Wrong plan executed confidently.
- Tool misuse.
- Hidden side effects.
- Incomplete verification.
- Sunk-cost bias toward bad generated work.
- Concurrent agent changes conflicting with humans.

Osmani highlights additional risks: coherent incorrectness, environment drift, async coordination problems, review bottlenecks, and cascading tool actions.

Human Oversight Checkpoints

Meaningful checkpoints happen before irreversible or high-risk action. For isolated PR-producing agents, some edits may happen first, but merge, release, permission expansion, and exception decisions still require review.

Examples:

- Approve plan before edits.
- Approve file writes.
- Review failing tests before retry loops.
- Approve PR merge.
- Require human signoff for sandbox exceptions.

Testing Strategy for Agents

Agents need tests at multiple levels:

- Unit tests for tool dispatcher.
- Integration tests for tool-result handling.
- Golden examples for workflow output.
- Safety tests for denied operations.
- Manual review for architecture decisions.

Thomas and Hunt's testing guidance applies more strongly as autonomy increases: tests are feedback, property tests validate assumptions, and continuous testing catches regressions before they compound.

Feedback Loops With Agents

Autonomous systems need explicit feedback loops.

- Make the plan visible before execution.
- Make commands, diffs, and test output reviewable.
- Fail fast when the environment cannot run verification.
- Feed review comments back into the next agent run.
- Track when a PR should be fixed, rerun, or discarded.

Managing Concurrent Agents

Parallel agents can increase throughput and integration risk.

- Assign non-overlapping tasks where possible.
- Avoid multiple agents editing the same subsystem without coordination.
- Require small PRs and clear branch names.
- Rebase and rerun tests before accepting output.
- Keep a human owner for final integration decisions.

Lab 5 Final Connection

Your controlled workflow should demonstrate:

- Core Java agent loop.
- Connected tool declarations and dispatcher.
- Reproducible multi-step workflow.
- Human oversight checkpoint.
- Example run artifacts.

Final Milestone 1 Connection

The proposal and architecture due this module should show how Labs 1-5 become a team project.

Your architecture should identify:

- Input.
- Generation.
- Evaluation.
- Workflow.
- Persistence.
- Human oversight.

Practice Activity

Evaluate a candidate task for delegation.

Classify it:

- Safe to automate.
- Safe with checkpoint.
- Human-only decision.

Use a decision table with task scope, risk, available tests, required permissions, review owner, and evidence needed.

Takeaways

- More autonomy requires stronger controls.
- Plan-execute-verify-report must be visible.
- Human review remains central to agentic workflows.